Experiment No. 0

Aim

To understand and implement data handling, statistical analysis, normalization, and data visualization using NumPy, Pandas, Matplotlib, and Seaborn.

Objective

- Perform numerical computations using NumPy
- Handle datasets using Pandas
- Visualize data using Matplotlib and Seaborn
- Perform Exploratory Data Analysis (EDA)

1. Dataset Source

Student Performance Dataset – Kaggle
https://www.kaggle.com/code/shrouk595nasr/student-performance-dataset/input

2. Dataset Description

The dataset contains student academic performance data.

Features:

- Gender (Categorical)
- Race/Ethnicity (Categorical)
- Parental Level of Education (Categorical)
- Lunch (Categorical)
- Test Preparation Course (Categorical)
- Math Score (Numerical)
- Reading Score (Numerical)
- Writing Score (Numerical)

Dataset Size:

- ~1000 rows
- 8 columns

Target Variable:

Not applicable (EDA-based experiment)

3. Mathematical Formulation

Mean

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Standard Deviation

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$$

Min-Max Normalization

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Pearson Correlation

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

4. Algorithm Limitations

- No predictive modeling performed.
- Correlation only measures linear relationships.
- Min-Max scaling is sensitive to outliers.
- Visual interpretation may vary.

5. Methodology / Workflow

Dataset
↓
Data Loading (Pandas)
↓
Statistical Computation (NumPy)
↓

Normalization
↓
Visualization (Matplotlib & Seaborn)
↓
Analysis & Interpretation

6. Implementation with Code and Output

Exercise 1: NumPy Basics

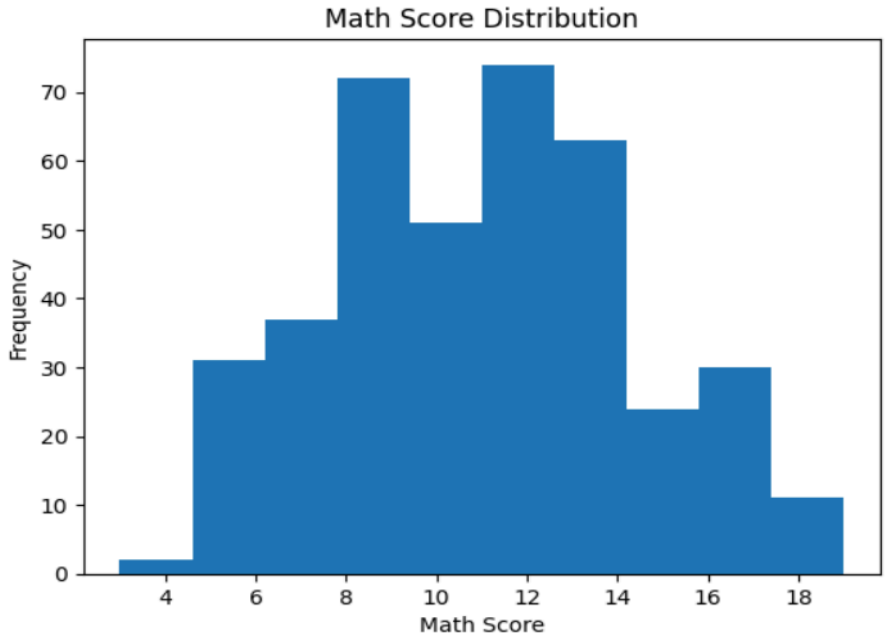
```
import numpy as np
import pandas as pd

df = pd.read_csv('/content/student_data.csv')

scores = df.iloc[:, -3:].values

mean_scores = np.mean(scores, axis=0)
median_scores = np.median(scores, axis=0)
std_scores = np.std(scores, axis=0)

print("Mean Scores:")
print(mean_scores)

print("\nMedian Scores:")
print(median_scores)

print("\nStandard Deviation:")
print(std_scores)

min_val = np.min(scores)
max_val = np.max(scores)

normalized_scores = (scores - min_val) / (max_val - min_val)

print("\nNormalized Scores (first 5 rows):")
print(normalized_scores[:5])
```

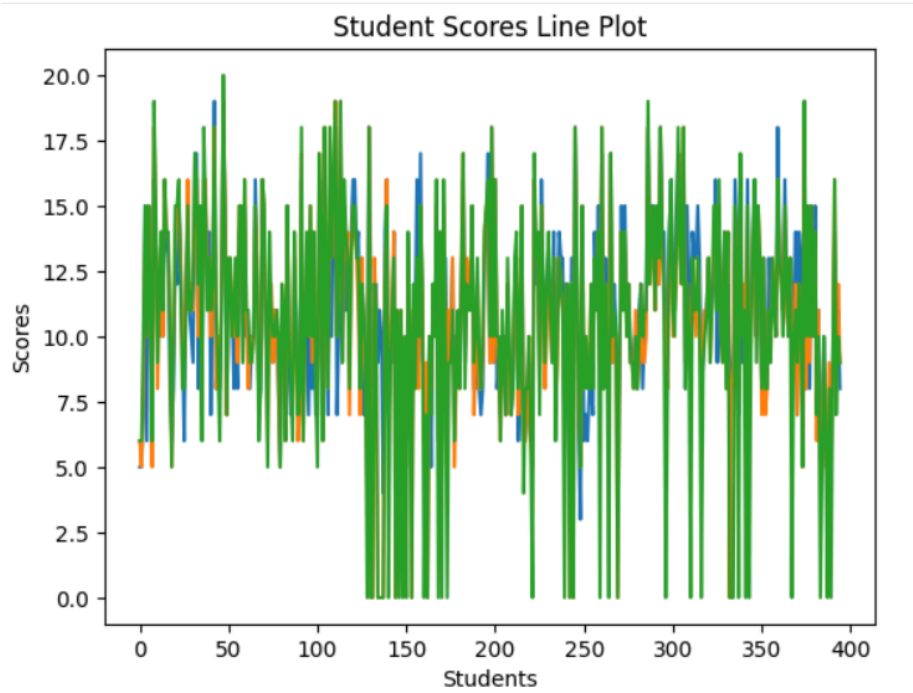
```
Mean Scores:
[10.90886076 10.71392405 10.41518987]

Median Scores:
[11. 11. 11.]

Standard Deviation:
[3.3149905  3.75674024 4.57563964]

Normalized Scores (first 5 rows):
[[0.25 0.3  0.3 ]
 [0.25 0.25 0.3 ]
 [0.35 0.4  0.5 ]
 [0.75 0.7  0.75]
 [0.3  0.5  0.5 ]]
```

Exercise 2: Pandas Data Handling & Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

scores = df.iloc[:, -3:]

plt.figure()
plt.plot(scores.iloc[:, 0])
plt.plot(scores.iloc[:, 1])
plt.plot(scores.iloc[:, 2])
plt.xlabel("Students")
plt.ylabel("Scores")
plt.title("Student Scores Line Plot")
plt.show()

plt.figure()
plt.hist(scores.iloc[:, 0], bins=10)
plt.xlabel("Math Score")
plt.ylabel("Frequency")
plt.title("Math Score Distribution")
plt.show()

plt.figure()
```

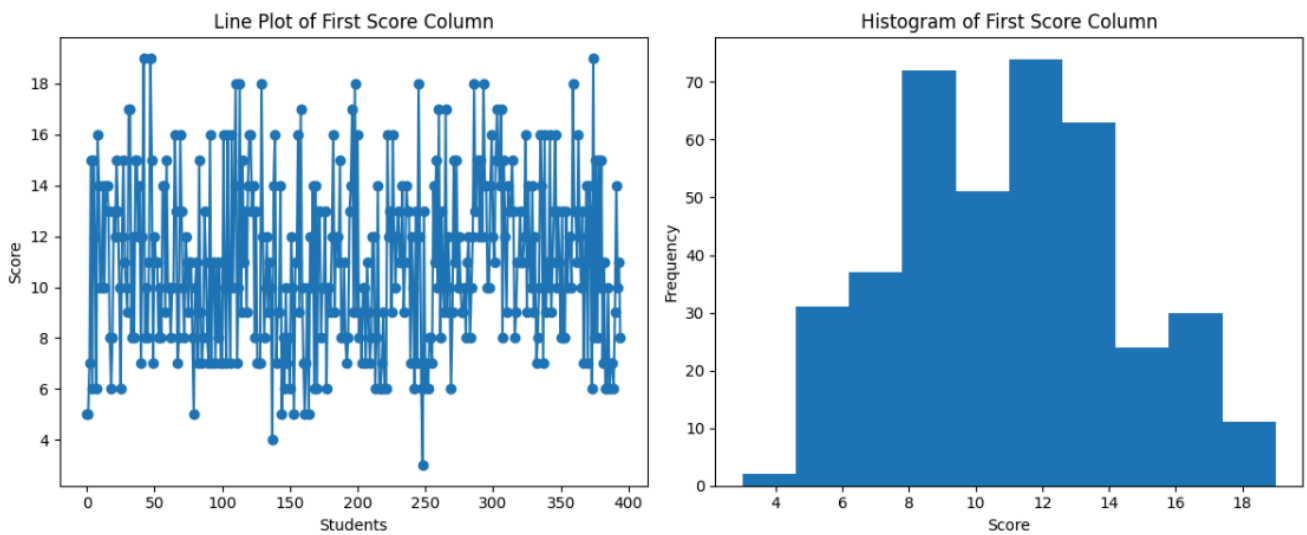
```
sns.boxplot(data=scores)
plt.title("Score Distribution Boxplot")
plt.show()

plt.figure()
sns.heatmap(scores.corr(), annot=True)
plt.title("Score Correlation Heatmap")
plt.show()
```





Exercise 3: Matplotlib Subplots

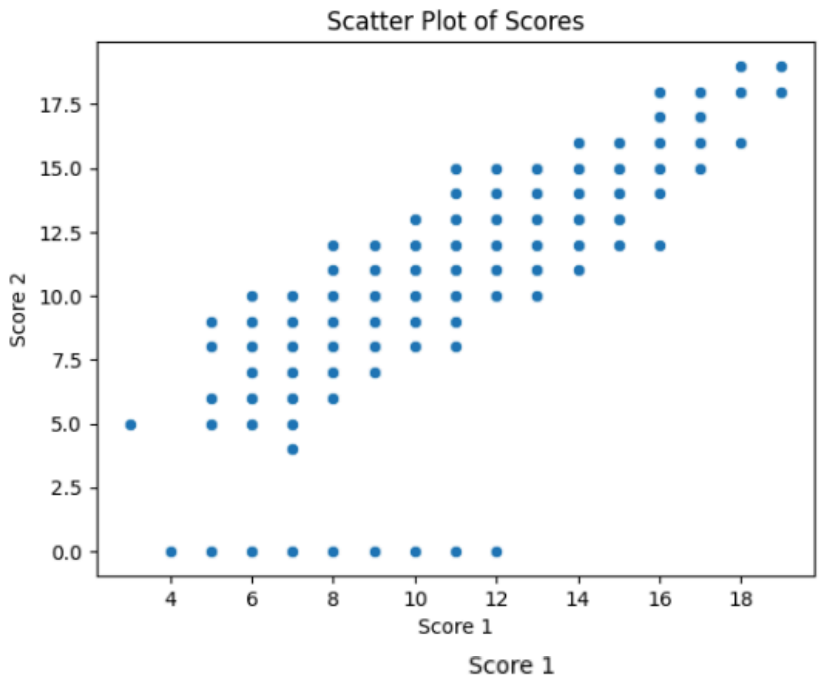
```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(scores.iloc[:, 0], marker='o')
plt.xlabel("Students")
plt.ylabel("Score")
plt.title("Line Plot of Math Score")

plt.subplot(1, 2, 2)
plt.hist(scores.iloc[:, 0], bins=10)
plt.xlabel("Score")
plt.ylabel("Frequency")
plt.title("Histogram of Math Score")

plt.tight_layout()
plt.show()
```

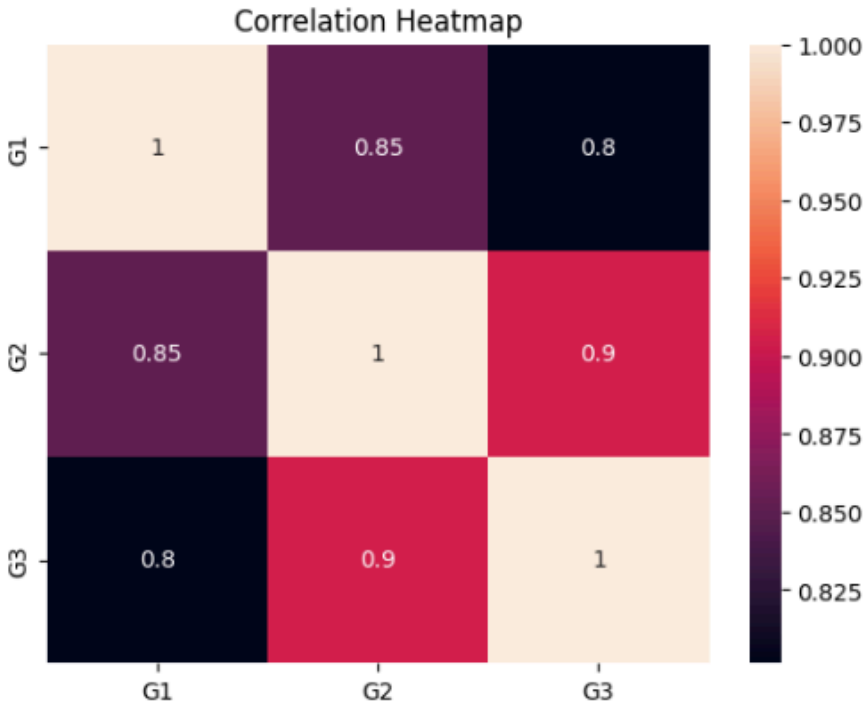


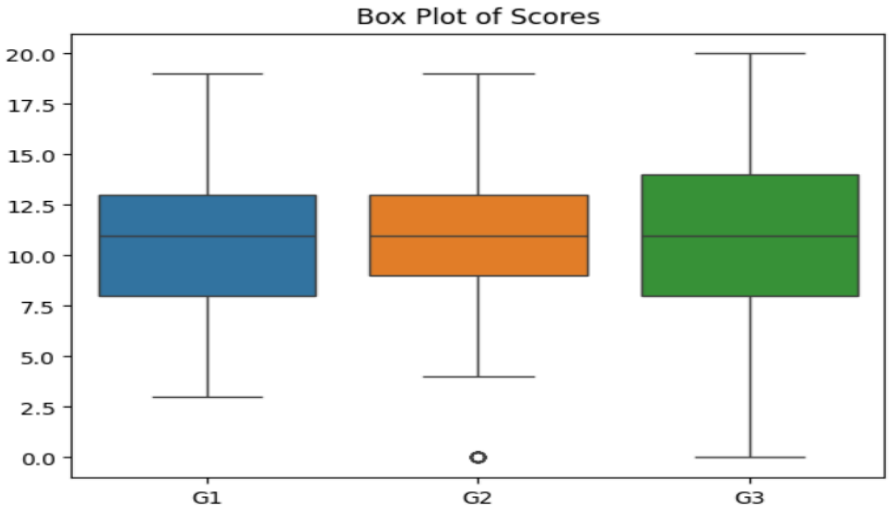
Exercise 4: Seaborn Visualization

```
sns.scatterplot(x=scores.iloc[:, 0], y=scores.iloc[:, 1])
plt.xlabel("Math Score")
plt.ylabel("Reading Score")
plt.title("Scatter Plot of Scores")
plt.show()

sns.heatmap(scores.corr(), annot=True)
```

```
plt.title("Correlation Heatmap")
plt.show()

sns.boxplot(data=scores)
plt.title("Boxplot of Scores")
plt.show()
```







7. Performance Analysis

Observations:

- Reading and Writing scores are highly correlated.
- Mean and median are close → distribution is fairly symmetrical.
- Standard deviation shows moderate spread.
- Few mild outliers observed.

8. Hyperparameter Tuning

No ML model is trained; hence:

- No algorithmic hyperparameters.
- Visualization parameters tuned:
  - `bins=10`
  - `figsize=(12,5)`
  - `annot=True` for heatmap

Changing these improves visualization clarity but does not affect dataset values.

Conclusion
This experiment provides hands-on experience with essential Python libraries used in Machine Learning. Students learn how to preprocess data, perform exploratory analysis, and visualize insights, forming a strong foundation for building ML models.